LiveRecall: Networking Design and Evaluation

1. Architecture and Protocol

1.1 System Architecture

The supplied caption_source.py emits full-text revisions over WebSocket. One asyncio server routes connections by their hello message. CaptionState retains latest active revisions and finalized IDs; History retains recent finals. Each client has a bounded live queue, sender and supervised session. **RecoveryPlan** supplies a bounded snapshot prefix to that sender. The browser is served separately over HTTP and maintains display/recall state. Lecture state survives source reconnects but is lost on server restart.

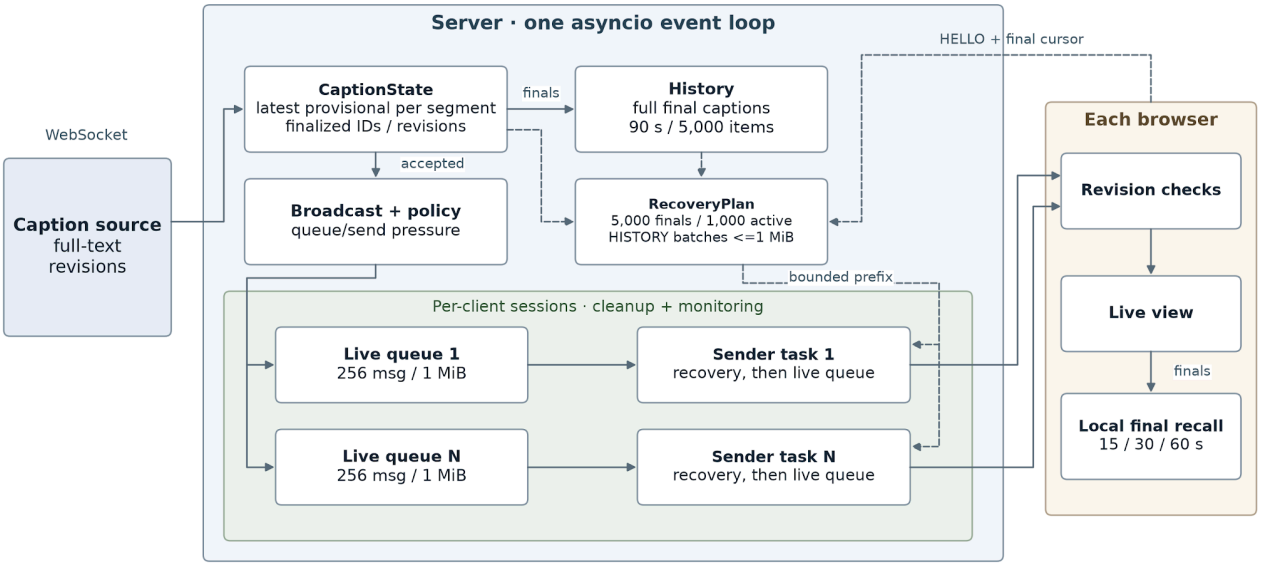


Figure 1. CaptionState stores active/latest revisions and finalized IDs; History stores full finals. RecoveryPlan supplies a bounded prefix directly to each sender; solid queue paths carry live updates.

1.2 Application Protocol

Messages are JSON text frames. SOURCE_HELLO identifies a source; CLIENT_HELLO supplies client_id, session, the last accepted final cursor and requested history window. CAPTION_UPDATE carries session, segment, revision, source_ts, final and full revision text. HISTORY contains oldest-first final items and may span multiple bounded batches following CLIENT_HELLO. Batches precede active snapshots and live updates. SOURCE_DONE is diagnostic; hello/session fields do not provide authentication or independent multi-lecture state.

2. Revision and Recovery Semantics

2.1 Revision State and Bounded History

The server accepts an active segment only at a **strictly newer revision** and **rejects all changes after finalization**. The browser independently applies equivalent revision/final rules. Server state caps active segments at **1,000** and remembered final revisions at **10,000**; a segment watermark rejects forgotten ranges. Full finals are retained separately for **90 seconds** and at most **5,000 items**. Retention uses server monotonic acceptance time, with pruning on insertion/query. Count limits can shorten retained context; batching avoids truncation at a total message-size budget.

2.2 Local Recall and Browser Behavior

Live and recovered captions share **applyCaption()/acceptUpdate()**, so accepted revisions replace one segment line rather than adding duplicates. Provisional and final styling differs. The browser tracks at most **2,000 segments** and renders at most **300 lines**, with follow-live scrolling and a jump control. Accepted finals enter a local rolling recall store, filtered by source timestamps: default 30 seconds,

selectable **15/30/60 seconds**. Normal recall reads this store **without a server request**. This timestamp comparison assumes synchronized source/browser clocks.

2.3 Reconnect and Late-Join Recovery

On reconnect, the browser retries after three seconds and reports its last accepted final; page refresh resets caption state. The server validates the cursor/window (at most 60 s), snapshots retained finals and latest active revisions, and registers **without yielding**. **RecoveryPlan** preflights every item, shares at most 5,000 final/1,000 active strings, and streams HISTORY batches **≤1 MiB**, then active updates, before the live FIFO. Only one batch is serialized at a time. Oversized individual items fail before partial delivery; expired/count-pruned history, live overflow and disconnects still limit recovery.

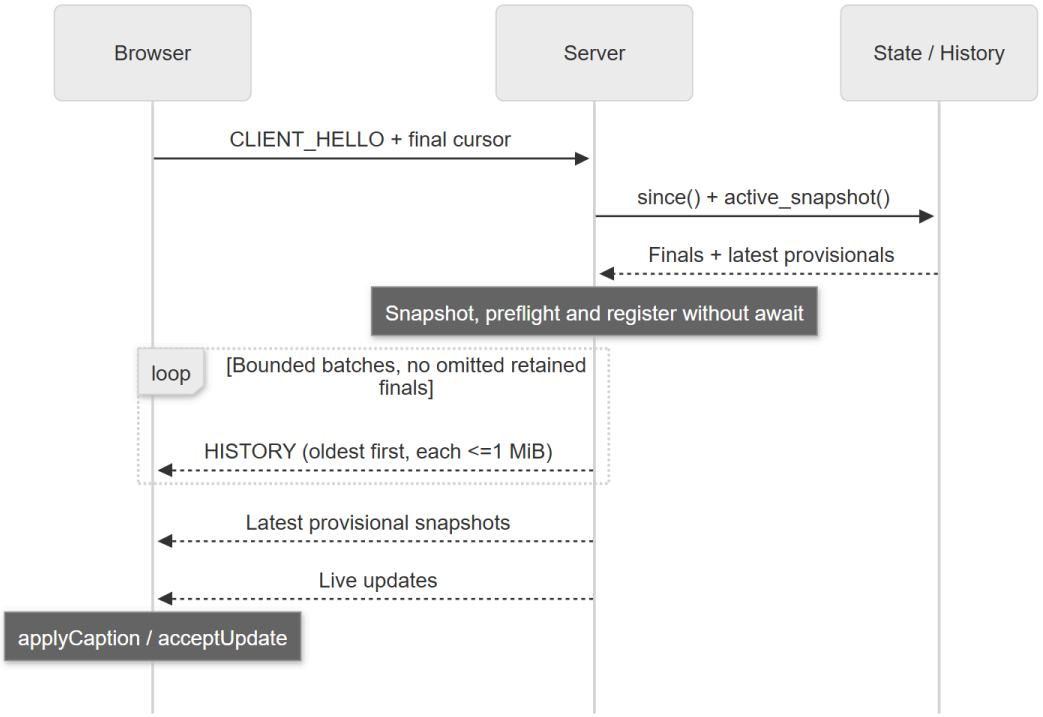


Figure 2. Snapshot and registration do not yield; the sender streams HISTORY batches, active state, then live updates.

3. Per-Client Buffering and Overload Handling

3.1 Queues, Senders and Session Lifecycle

Each connection owns a bounded FIFO, one asyncio sender and a supervised session. The producer enqueues **without awaiting client socket writes**, isolating normal receivers from blocked sends. Defaults cap queued live payloads at **256 messages and 1 MiB**; send and close attempts have **10 s and 3 s** deadlines. Session teardown unregisters the connection, clears queued data, **cancels and awaits child tasks**, and removes monitoring/policy/eviction entries.

3.2 Monitoring and Overload Policy

Monitoring separates queued count/bytes/age, the in-flight payload, successful local sends and transport-buffer bytes. The current producer enables semantic coalescing when its per-client policy detects queue/send pressure. Degraded mode starts at **50% queue utilization** or **0.5 s queue/send age**; exit requires utilization **≤20%** and ages **≤0.15 s**. **Coalescing never removes queued finals**, but **teardown and recovery limits can still prevent delivery**. Obsolete provisionals may be removed. The revised cleanup removes a provisional only when a higher revision of the same segment is queued or being admitted; **each active segment's latest version is preserved**. If admission still fails after semantic dropping, the server **unregisters the client and schedules a close with code 1013**. Keepalive uses a 2 s ping interval and 5 s pong timeout; disconnected clients resume from their last accepted final cursor. This is timeout-driven recovery, not a precise measurement of displayed-caption lag.

4. Experimental Evaluation

4.1 Experimental Setup and Metrics

- **Host:** GB10; Ubuntu 24.04.4 LTS, ARM64, 20 logical CPUs.
- **Runtime:** Python 3.12.3 and websockets 16.1.1.
- Version: tested recovery export f630bc443b7a (SHA-256 source manifest); historical T3 baseline acef9a3.
- **Execution:** unchanged public harness, source seed 3103, sequential runs and a fresh server for each repetition.
- **Aggregation:** T1–T5 each use **three runs per configuration** and report **medians of per-run metrics**. T3 whiskers show observed min/max. The separate 300 s RSS observation is a single supplementary run.
- **Network:** source, server and simulated clients run on same-host loopback. T1–T4 and RSS use no impairment; T5's isolated namespace and delay/loss profile are described in Section 4.6.

Metrics: Caption Age is client receive time minus source_ts; P50/P95/P99 and Fresh Caption Ratio (T=500 ms) use accepted live updates. Final Caption Completeness divides unique received/recovered finals by generated finals; Fresh Final Caption Ratio uses generated finals as denominator. **Normal Update Coverage divides distinct received live revisions by generated revisions.** Public HISTORY-response time ends at HISTORY receipt; oracle Recovery time ends only when **all required finals and active state** arrive (Appendix B).

Table 1. T2/T4/T5 medians of three runs. Fresh means age ≤500 ms; final completeness includes recovery. All runs recorded zero duplicate/stale captions and connection errors.

Test	P50 (ms)	P95 (ms)	P99 (ms)	Live fresh (%)	Fresh finals (%)	Final complete (%)	Normal Update Coverage (%)
T2	3.04	5.46	6.47	100.00	100.00	**100.00**	100.00
T4	2.81	4.96	5.65	100.00	98.42	**100.00**	98.25
T5	258.11	663.24	1006.41	90.76	89.43	**100.00**	99.98

4.2 T1: Basic Correctness

- **Workload:** 20 clients × 20 s × 3 runs; 22 final segments per run.

Results: P50/P95/P99 = 1.73/2.50/2.89 ms; Fresh Caption Ratio, Final Caption Completeness and **Normal Update Coverage** were **100.00%**. All runs recorded zero duplicate/stale captions and connection errors. This validates the basic public workload, not browser rendering or adversarial revision order.

4.3 T2: Public Scale

- **Workload:** 100 clients × 40 s × 3 runs; 49 final segments per run.

Table 1 summarizes the public-scale results. All source revisions and finals reached all 100 simulated clients; the results apply to this loopback workload.

4.4 T3: Slow-Client Isolation and Before/After Comparison

- Comparison: acef9a3 versus recovery export f630bc443b7a; SHA-256 matched the unchanged public source, runner, clients and analyser.
- **Workload:** 100 clients × 60 s × 3 runs per version; 10 slow readers wait 500 ms per receive; 76 final segments/228 revisions per run.
- **Groups:** 90 normal and 10 slow clients; results are reported separately.

Table 2. Accepted live-caption latency and freshness; unique final completeness includes HISTORY recovery. Fresh means age ≤500 ms. Normal Update Coverage counts received revisions, not recovered finals.

Version / group	P50 (ms)	P95 (ms)	P99 (ms)	Fresh (%)	Final (%)	Coverage (%)
Before / normal	2.93	5.52	6.00	100.00	100.00	100.00
Before / slow	14,712.05	28,076.50	29,127.81	1.69	51.32	51.75
Current / normal	2.75	3.84	4.62	100.00	100.00	100.00
Current / slow	6,960.66	13,359.42	13,808.61	2.65	**100.00**	49.56

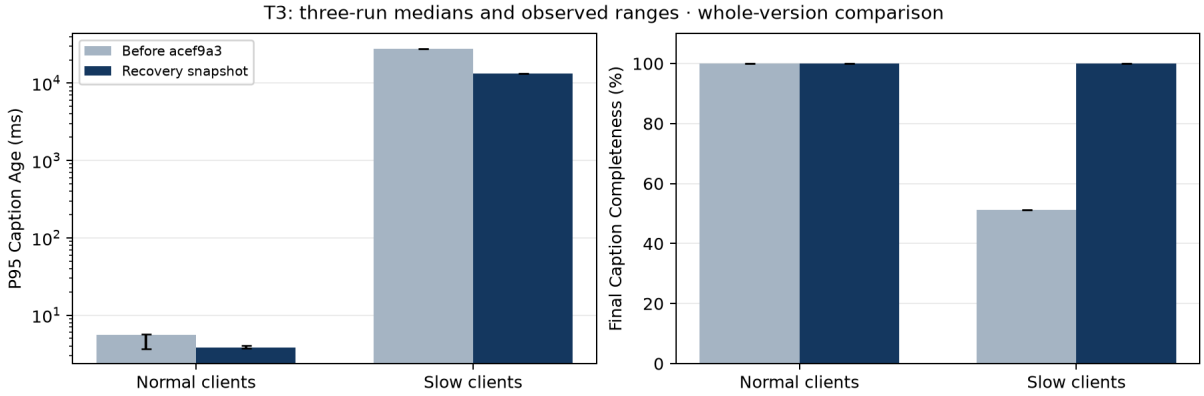


Figure 3. Normal/slow T3 latency and final completeness before and after the full version change. Latency uses a log scale and excludes HISTORY items; the accepted-live sample populations differ between versions.

Normal clients retained 100% **Normal Update Coverage** and Final Caption Completeness in both versions. Slow-client P95 changed from 28.08 s to 13.36 s; final completeness changed from 5**1.32%** to **100.00%**. Current runs still recorded **20 CONNECT_ERROR** events and 380 recovered items per run (medians), with zero duplicate/stale captions. Slow-client Fresh Final Caption Ratio remained **1.32%**: eventual completeness did not provide timely delivery. Median HISTORY-response P95 was 2.41 ms, not complete recovery time. This compares whole default versions, not an isolated optimization.

4.5 T4: Reconnect and Recovery

- **Workload:** 100 clients × 60 s × 3 runs; 20% disconnect at 15 s for 5 s; 76 finals per run.

T4 recovered 140 HISTORY items through 20 reconnects; connection-to-HISTORY P95 was 3.34 ms. This is not full recovery time. The separate oracle checked **all required finals and active state**: large-history median **16.34 ms** (Appendix B). Offline provisionals explain lower coverage in Table 1.

4.6 T5: Network Impairment

- **Network:** isolated Linux network namespace; supplied delay_loss.sh on lo (**120 ms ±30 ms normal jitter, 3% loss**). The source-to-server and server-to-client paths both traverse this interface; these are not end-to-end one-way delay values. Profile reset after all runs.
- **Workload:** 100 clients × 40 s × 3 runs; 49 finals per run.

All finals arrived, but not all live updates met the 500 ms target (Table 1). This is consistent with TCP retransmission, ordered delivery and jitter; without packet traces, their individual effects are not isolated. Bandwidth halving was not tested. Appendix A shows the receive-age distributions.

4.7 Server Memory Observations

- **Method/metrics:** server-child VmRSS sampled every 0.5 s, tracked by PID; compare three T3 runs/version and one additional 300 s current run with the same 90/10 client mix. Report peak, early/late median and post-60 s slope.

An external observer sampled server-child VmRSS every 0.5 s. Peak T3 RSS ranged from 31.93–31.99 MiB before to 33.36–33.38 MiB for the recovery export (three runs each). A separate 300 s workload produced a 308.9 s trace including startup/teardown: peak **36.48 MiB**, median 32.14 MiB during seconds 10–30 and 36.46 MiB in the final 30 s. The post-60 s slope was about **0.70 MiB/min**. These data **do not establish a plateau**, leak freedom or long-term boundedness.

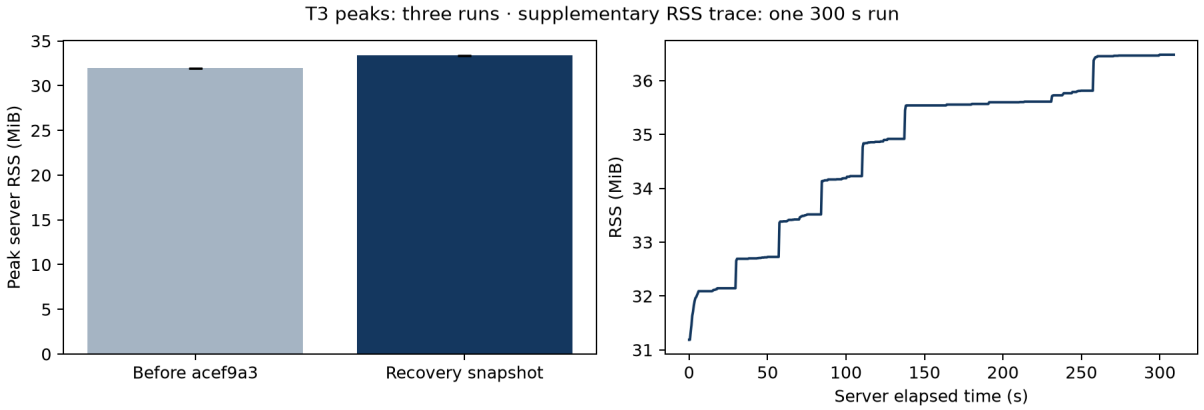


Figure 4. Peak server RSS across three T3 runs (median and min/max), and the recovery export's supplementary 300 s workload trace. Sampling includes startup/teardown; RSS excludes clients and kernel socket buffers. One long run provides no confidence interval.

5. Failures, Trade-offs and Limitations

The revised source passed **141 Ubuntu tests** and the oracle recovered every required item in all 15 runs. The separate 108-case simulation found no useful-state drops after the cleanup fix. T3 slow-reader freshness remains poor. Live-queue overflow can still evict clients and discard pending finals; batching **does not fix that policy**. History count limits can shorten the 60 s recoverable window under bursts. Tests do not establish arbitrary bandwidth-shaped, larger-scale or long-term memory behavior. The single 300 s RSS observation is supplementary.

6. Future Work and FYP Direction

Next steps are final-aware overload scheduling, retention-capacity validation for the full 60 s window, constrained-bandwidth tests and receiver-aware lag measurements. Longer studies should track queue/state/task counts and process/kernel memory.

References

[1] CS3103 Assignment 4: LiveRecall handout, updated 17 October 2026, sections 8, 10 and 12.

[2] Repository snapshots acef9a3 and 18f0dbf; tested recovery export f630bc443b7a with source hashes. Raw results and reproducible scripts: experiments/delivery_20261103/ and experiments/recovery_batches/.

Appendix A. Additional Experimental Figures

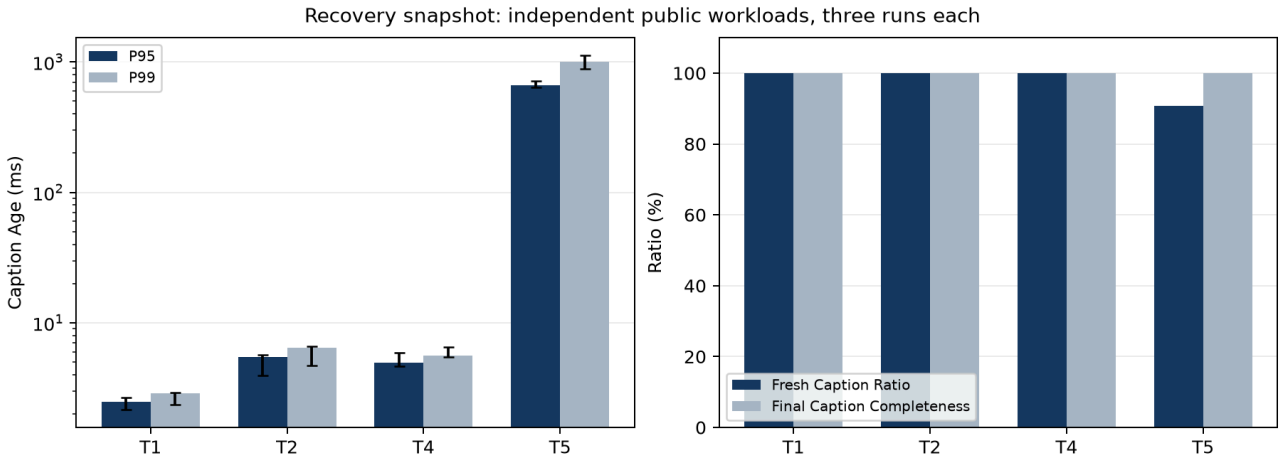


Figure 5. T1/T2/T4/T5 medians of three per-run metrics; whiskers show observed min/max, not confidence intervals. Latency excludes HISTORY; finals include recovery. T3 is shown separately in Figure 3. Workloads differ, so this is not a controlled scaling curve.

Unshaped T1/T2/T4 retain 100% live freshness, while T5 delay/loss increases tail age and lowers freshness despite **100% final completeness**. T4's offline period reduces fresh-final ratio because recovered finals are older than the 500 ms target.

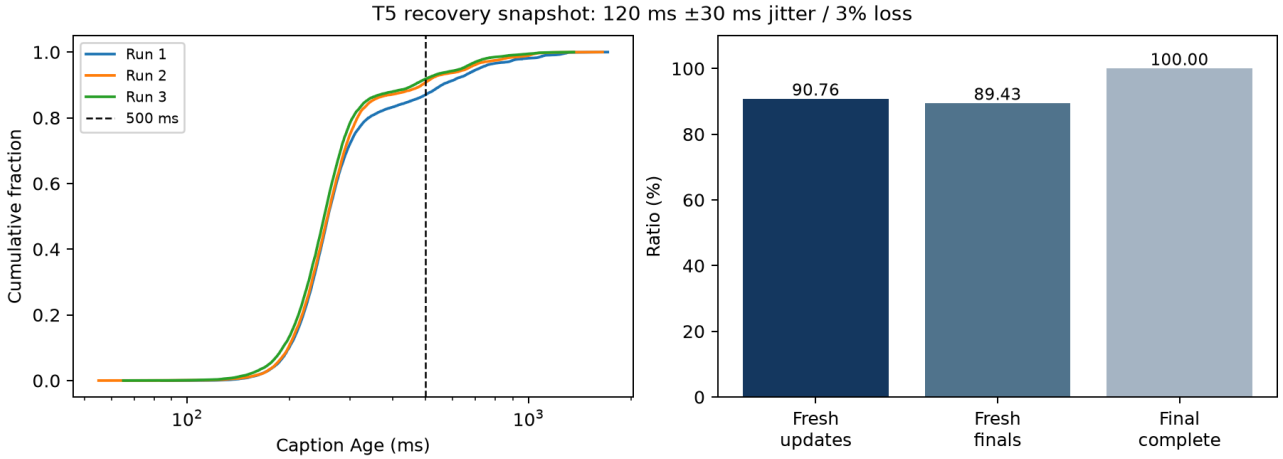


Figure 6. T5 per-run empirical cumulative live-age distributions and freshness/final completeness, using the accepted three-run batch. Each curve includes all accepted live updates from one run; the dashed line marks 500 ms. Latency uses a log scale.

All three runs converge to 100% Final Caption Completeness, while about 91% of live updates arrive within 500 ms. Receive-age tails show that eventual reliability and timely delivery are separate properties. The fixed source seed does not fix netem's random loss/jitter.

Appendix B. Complete Recovery Verification

Method: GB10 loopback, Python 3.12.3, no network impairment; five cases × three fresh connections per version. Before is 18f0dbf; batched recovery is source snapshot f630bc443b7a (recorded source hashes). The independent oracle checks every required final and latest active payload, duplicates and ordering. Completion runs from successful connection until all required state arrives; first-HISTORY response is recorded separately.

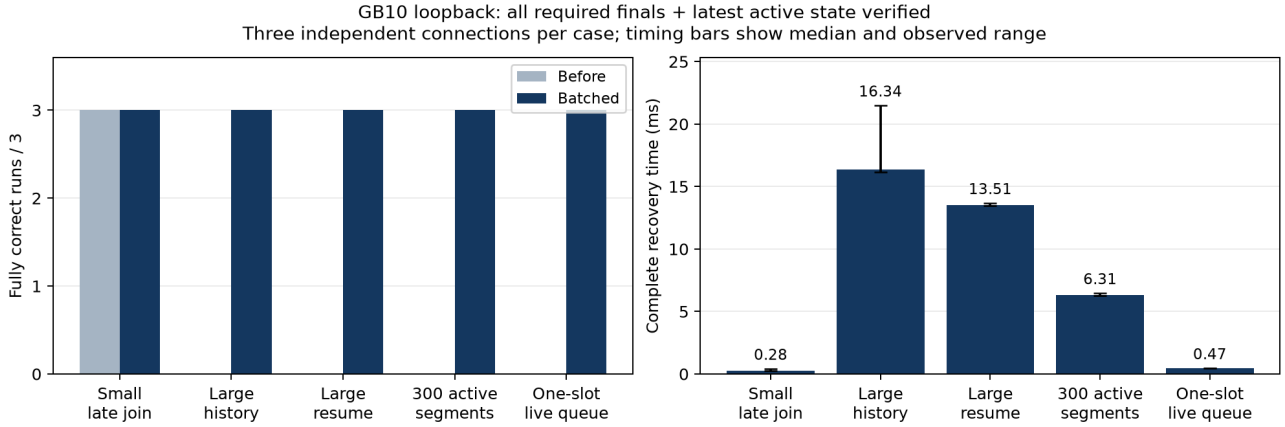


Figure 7. Oracle-verified complete recovery. Bars show successful runs out of three and batched completion medians with observed min/max. Failed pre-fix runs have no completion time, rather than zero latency.

- Small late join: 20 finals + 3 active; both versions 3/3 correct; batched median 0.28 ms.
- Large history: 96 finals of approximately 20 KB each + 3 active; before 0/3, batched 3/3; **16.34 ms**, two HISTORY batches.
- Large resume: 16 known finals excluded; before 0/3, batched 3/3; 13.51 ms, two batches.
- Many active: 10 finals + 300 active, exceeding the 256-slot live queue; before 0/3, batched 3/3; 6.31 ms.
- One-slot live queue: 10 finals + 16 active; before 0/3, batched 3/3; 0.47 ms.

The pre-fix implementation rejected the larger/capacity-constrained contexts before delivery. Every batched run returned all expected state with zero duplicates. This validates the tested recovery boundary, not arbitrary congestion or hidden workloads. Expired/count-pruned history, individually oversized messages and live-queue overflow remain limits. Methods, source hashes and per-run results: experiments/recovery_batches/.